

- レンダリングのためにウィジェットを Flutter に送り返す.
- Flutter は return を使って UI 要素を構築し,動的に表示する.

b. return で返されるウィジェットの理解

ウィジェット	使用目的	例
MaterialApp	ナビゲーション, テーマ, ローカライゼーションを提供するルートウィジェット	<code>return MaterialApp(...)</code>
Scaffold	AppBar や body などを含むメイン画面の構造を定義する	<code>return Scaffold(...)</code>
AppBar	アプリのタイトルや操作を表示する上部ナビゲーションバー	<code>AppBar(title: Text('My App'))</code>
Text	画面にシンプルなテキストを表示する	<code>Text('Hello, world!')</code>

- ポイント:
 - ◇ Flutter では,build() 内で return を使ってウィジェットを返さないと画面に表示されない.
 - ◇ return がないと,Flutter は何を表示すべきか判断できない.

c. 比較: Java における return の使われ方

- Java も Flutter (Dart) も,return を使って作成したオブジェクトを返す.
- Java では return は View や UI コンポーネントを返すことが多い.
- Flutter では,return はよりシンプルでウィジェット中心の設計になっている.
- 主な違い:

機能	Java (Android)	Flutter (Dart)
戻り値の型	View (TextView,Button など) を返す	Widget (Text,Scaffold など) を返す
UI の構造	UI は通常 XML で管理	Dart 内でウィジェットを使って UI を構築
柔軟性	View の階層を変更する必要がある	build() メソッド内で簡単に管理可能

ステップ 8 : Flutter の MaterialApp とは ?

a. MaterialApp とは ?

- MaterialApp は,Google の Material Design に準拠した Flutter アプリの基盤となる.
- アプリの基本構造とビジュアルスタイルを提供する.
- テーマ,ナビゲーション,デフォルトの動作を設定し,スムーズなユーザー体験を実現する.

- MaterialApp の主な機能:
 - ◇ ビジュアルスタイルの定義 (テーマ,色,フォント) .
 - ◇ 異なる画面間のナビゲーション管理.
 - ◇ デフォルトの動作を提供し,開発を簡単にする.

b. MaterialApp の基本的な例

- Flutter のコード例:

```
MaterialApp(
  home: Scaffold( // The main layout for the screen
    appBar: AppBar( // The top bar with a title
      title: Text('Welcome to Flutter!'),
    ),
    body: Center( // The main content in the middle of
the screen
      child: Text('Hello, Flutter!'),
    ),
  ),
);
```

- MaterialApp の構造の分解・解説

プロパティ	機能	例
home	アプリ起動時に最初に表示される画面を定義する	home: Scaffold(...)
Scaffold	AppBar や body を含む主要なレイアウトを提供する	Scaffold(appBar: ..., body: ...)
AppBar	タイトル付きの上部ナビゲーションバーを表示する	AppBar(title: Text('Welcome'))
body	画面のメインコンテンツ領域を定義する	body: Center(child: Text('Hello, Flutter!'))

- 説明:
 - ◇ home → アプリ起動時に最初に表示される画面.
 - ◇ Scaffold → 画面の基本的なレイアウトを提供する (AppBar,body を含む) .
 - ◇ AppBar → タイトルやボタンを配置できるトップバー.
 - ◇ body → メインコンテンツ (テキスト,画像,ボタンなど) .

c. なぜ MaterialApp が重要なのか?

- アプリの基本構造を提供し,ゼロから作る手間を省く.
- Google の Material Design に準拠し,モダンで統一感のあるデザインを提供.

- 異なる画面間のナビゲーションを簡単に整理できる.
- MaterialApp の主なメリット:
 - ◇ すぐに使える構造 → 開発時間を短縮.
 - ◇ Material Design 準拠 → UI の一貫性を確保.
 - ◇ 簡単なナビゲーション管理 → 複数画面の管理が容易.
- d. 比較: Java におけるアプリのセットアップ方法
 - Java (Android 開発) では,UI コンポーネントは通常 XML ファイルで作成される.
 - Flutter では,MaterialApp とウィジェットを使って UI を宣言的に構築する.
 - Flutter のアプローチは,コードの複雑さを減らし,保守性を向上させる.

```
public class MainActivity extends AppCompatActivity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
    }
}
```

- 主な違い:

機能	Java (Android)	Flutter (Dart)
UI の構造	XML ファイルでレイアウトを定義	Dart コード内で直接 UI を作成
ナビゲーション設定	Intent や Fragment を使用	MaterialApp で簡単に管理
複雑さ	コードが長くなりやすく, 手動更新が必要	シンプルで宣言的な UI モデル

ステップ 9 : Flutter の Scaffold とは?

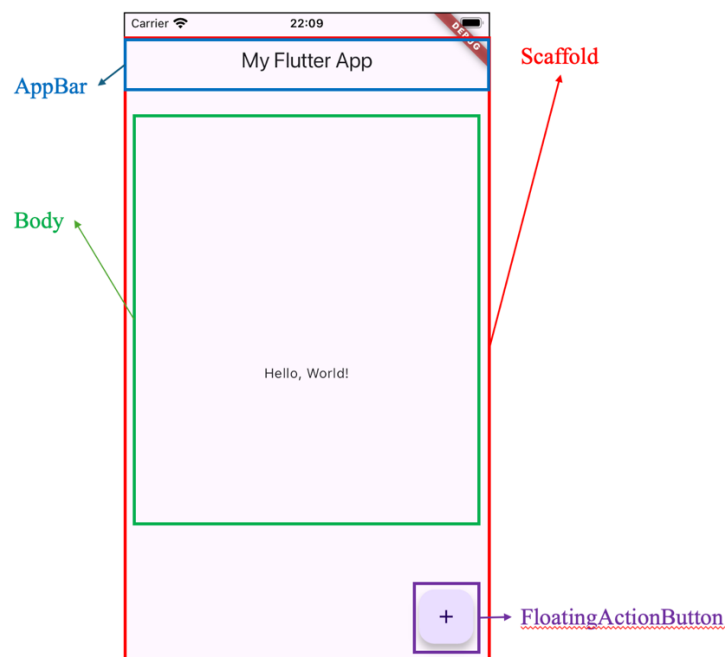
- a. Scaffold とは?
 - Scaffold は,Flutter アプリの画面の基本構造 (スケルトン) を提供する.
 - AppBar,body,FloatingActionButton などの主要な UI 要素を整理する.
 - UI レイアウトを簡素化し,アプリのインターフェースを構築しやすくする.
 - Scaffold の主要な UI コンポーネント:
 - ◇ AppBar → タイトルやナビゲーションボタンを表示するトップバー.
 - ◇ body → 画面のメインコンテンツ領域.
 - ◇ floatingActionButton → すぐにアクションを実行できるフローティングボタン.
- b. Scaffold の基本的な例
 - Flutter のコード例:

```
Scaffold(
  appBar: AppBar(
```

```

    title: Text('My Flutter App'),
  ),
  body: Center(
    child: Text('Hello, World!'),
  ),
  floatingActionButton: FloatingActionButton(
    onPressed: () {
      print('Button Pressed!');
    },
    child: Icon(Icons.add),
  ),
);

```



■ Scaffold の主要な構成要素

◇ appBar – 上部のナビゲーションバー

- ナビゲーションとタイトルを表示するトップバーを作成する.
- タイトル (Text) やアクションボタンを含むことが多い.
- appBar プロパティ内で AppBar ウィジェットを使用する.

◇ body – メインコンテンツエリア

- 画面の中で最も大きな部分で,ほとんどのウィジェット (テキスト,画像,ボタンなど) を配置する.
- Center, Column, Row などのウィジェットを使ってレイアウトを整理する必要がある.
- Text, ListView, GridView などのウィジェットを含むことが多い.

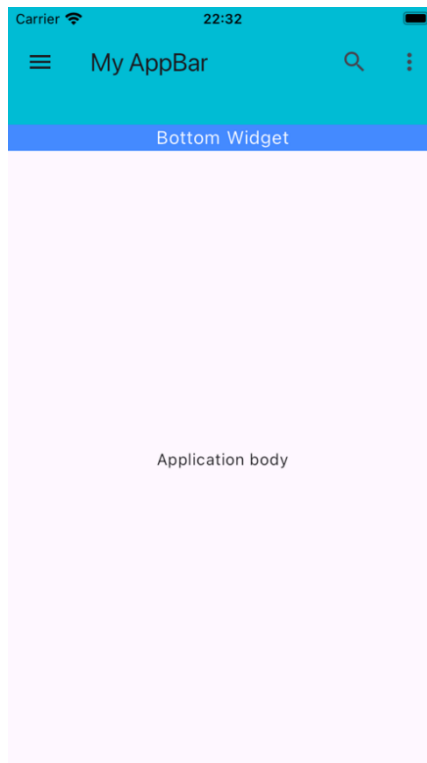
◇ floatingActionButton – クイックアクションボタン

- コンテンツの上に「浮く」ボタンで,クイックアクションに使用される.
- アイコンや他のウィジェットを内部に配置する必要がある.

- アイテムの追加やコンテンツのリフレッシュなどのアクションに使われる。
- c. なぜ Scaffold が重要なのか？
 - 構造化されたレイアウトを提供し,画面を作成しやすくする.
 - アプリの異なる部分で一貫した UI 体験を実現する.
 - MaterialApp と統合し,スムーズなナビゲーションとテーマ管理を可能にする.
 - Scaffold のメリット:
 - ◇ 適切な構造を提供 → UI デザインを簡素化する.
 - ◇ ユーザー体験を向上 → モダンでレスポンスなレイアウトを実現.
 - ◇ Material Design と統合 → UI の一貫性を確保する.

まとめ

- Flutter アプリは main() と runApp() から始まり,MyApp ウィジェットを読み込む.
- MyApp は StatelessWidget を継承し,ルートウィジェットである MaterialApp を返す.
- MaterialApp の中で Scaffold が使用され,画面レイアウトが構築される.
- build() メソッドはウィジェットツリーを構築し,return によって UI が定義される.
- AppBar,body,floatingActionButton などの主要なウィジェットは Scaffold の中に配置される.
- Flutter のウィジェットシステムは階層構造になっており,すべてがウィジェットの中に構築されている.



d. AppBar を使うべき場面

- AppBar がよく使われる場面:
 - ◇ アプリの統一感のあるトップバーが必要なとき.
 - ◇ タイトルやナビゲーションボタンを追加したいとき.
 - ◇ 検索,設定,その他のオプションを提供したいとき.
 - ◇ タブや追加コンテンツを AppBar の下に表示したいとき.

Flutter の Text ウィジェット

a. Text ウィジェットとは？

- Flutter アプリでテキストを表示するウィジェット.
- プレーンテキストとスタイル付きテキストの両方をサポート.
- 静的メッセージ,動的更新,レスポンスデザインに対応可能.

b. 主なプロパティ

- 主なプロパティ：

プロパティ	説明	使用例
data	表示するテキスト（必須）.	'Hello, Flutter!'
style	テキストの見た目を変更.	TextStyle(fontSize: 18, color: Colors.blue)

textAlign	テキストの配置(left, center, right).	TextAlign.center
maxLines	最大行数を制限.	maxLines: 2
overflow	テキストのオーバーフロー処理 (ellipsis, fade).	TextOverflow.ellipsis
softWrap	テキストの折り返し設定.	softWrap: true

c. 例：FlutterでのTextスタイルの設定

■ 例のFlutterコード：

```
Text(
  'Welcome to Flutter!', // The text to display
  style: TextStyle(
    fontSize: 20,          // Sets the font size
    color: Colors.green,   // Text color
    fontWeight: FontWeight.bold, // Makes the text bold
  ),
  textAlign: TextAlign.center, // Centers the text
  maxLines: 2,                // Limits to 2 lines
  overflow: TextOverflow.ellipsis, // Adds "..." if the
text is too long
)
```

■ 動作説明:

- ◇ 'Welcome to Flutter!' → 表示するテキスト.
- ◇ style → サイズ,色,太さなどを指定.
- ◇ textAlign: TextAlign.center → テキストを中央揃え.
- ◇ maxLines: 2 → 最大2行までに制限.
- ◇ overflow: TextOverflow.ellipsis → 長すぎる場合“…”を表示.

d. なぜTextウィジェットを使うのか？

■ Textウィジェットを使う理由:

- ◇ 静的・動的テキストの表示が簡単.
- ◇ 整列・折り返し・オーバーフロー処理が可能.
- ◇ どんなレイアウトやUIデザインにも適用できる.

FlutterのContainerウィジェット

a. Containerウィジェットとは？

- スタイル,位置,レイアウトを設定できるカスタマイズ可能な「ボックス」.
- 他のウィジェットをラップし,間隔,色,装飾を適用するのによく使われる.
- Flutterで最も柔軟で一般的に使用されるウィジェットの1つ.